Lab2 -Challenge

Wednesday, February 4, 2026 11:11 PM

5 Challenge

5.1 Aims

The challenge is to implement additional instruction(s) and/or hardware that can speed up EEP1 register multiplication. You are limited to no using more than 64 full adders: for example 8 Issie adder blocks of 8 bits each.

Note that if you add multiple copies of a design sheet containing new adders to your design each copy counts separately.

Credit can be obtained from any of the following:
1. The solution itself
2. Comparing the speed of your solution in clock cycles, and its cost in hardware adders, with the "pure software" implementations in Lab2.
3. Knowing how your solution can be used in different contexts: for example, to implement signed and unsigned multiplication.

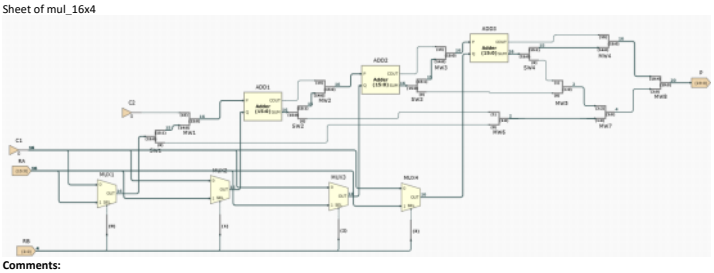
5.2 Design Notes

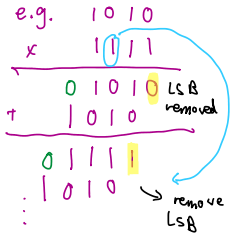
Speeding up EEP1 multiplication requires the definition of **additional ALU instructions in the ISA**. This is possible because some combinations of bits in the machine work are not currently used. Specifically the MOV instruction does not use register C, the three bits specifying this are set to 0 for a normal MOV.

5.2.1 Implementing unused instructions

There are 7 *additional* MOV instructions, using machine code as for MOV with nonzero in the c register field INS(4:2), can be used. The assembler recognises MOVCn Ra Rb where *n* = 1..7 and generates the correct machine code. You may use any of these instructions in Lab 2 to interface with additional hardware. For example MOVC1 Ra, Rb could be used to implement Ra := Ra op Rb, where op is some new operation you have defined.

Sheet of mul_16x4



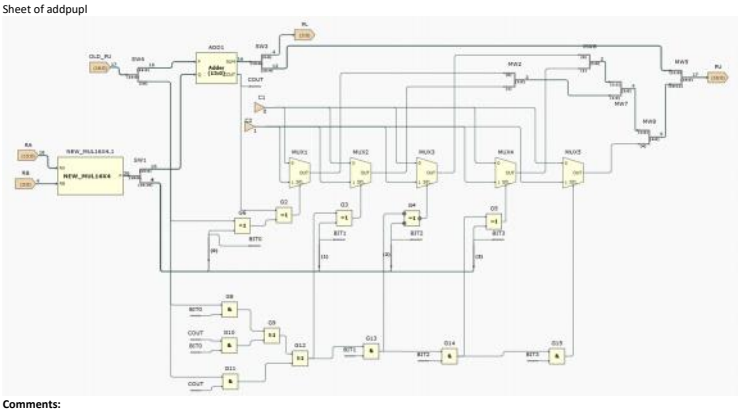


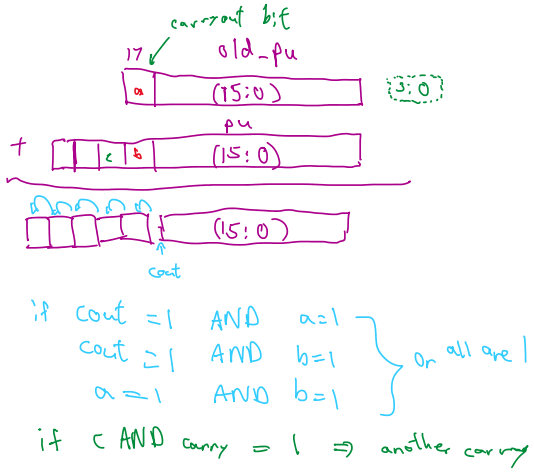
Comments:
Each bit of Rb would be used to select whether the output of the 2_MUX would be 0 or Ra based on whether the bit is 0 or 1. The 16-bit adders sum the first 15 bits of the previous sum with the Ra multiplied with a bit of Rb because in multiplication, as the bits of Rb go from bit 0 to bit 3, there is an extra 0 added to Ra, making it 17 bits. Hence, to use the 16-bit adders, I modified the 17 bit output from the previous addition.

I removed the LSB of previous sum and make COUT the MSB of the number. This way, we don't need to add an extra bit of 0 to Ra in order to achieve shifting. In the end, the most and least significant bits are merged together to form a 20-bit number.

The basic design logic is displayed on right hand side annotation

Sheet of addpupl

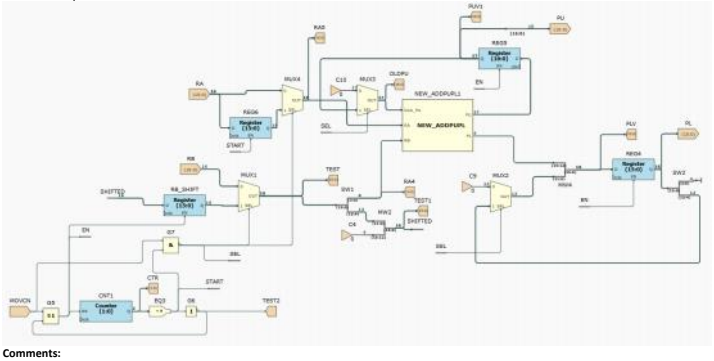


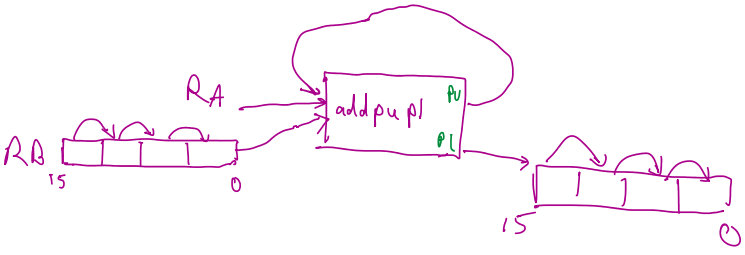


Comments:
This sheet adds the product of the current 4 bits with Ra and previous product of the less significant 4 bits of Rb with Ra.Then I split the sum into 17 bit of upper product and 4 bit of lower product

I made the upper product 17 bits as I combined the COUT bit with the upper 16 bits of SUM to account for cases where there is a carry out.

Sheet of multiplier





Comments:
Logic
The illustration on the right basically highlight the whole design logic:

- The 16-bit Rb is inputted to a right-shift mechanism that keeps shifting the four 4-bit blocks of Rb, taking only the least significant block to be inputted into the addpupl sheet
- The addpupl sheet has 3 inputs:
  a. The old PU
  b. Ra
  c. Rb

  The sheet then outputs the upper 16 bits and lower 4 bits as outputs. The lower 4 bit PL is inputted into a mechanism that first combines it with the most significant 12 bits of previous PL number then shifts the number right by 4 bits until the operation is done.

When the MOVCN = 1, G5.output = 1 so RB_SHIFT, REG4, REG5, REG6 and the Counter are enabled:
=> the counter starts counting in next clock cycle
=> registers store the values with Q port being the value stored in next clock cycle

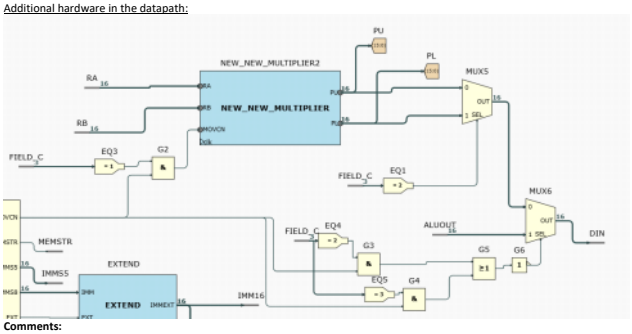
When the MOVCN instruction is detected, the counter still starts at 0 so MOVCN anded with EQ3 equals 1, making G7.output = 0:
=> at the first clock cycle, the value used is the RB value fed into the sheet
=> afterwards, it would be the value stored in REG6
=> the SEL which is G7.output ensures for MUX1, MUX2, MUX3 and MUX4 the initial value is correct

5.2.2 Adding to the datapath hardware

You can implement additional logic in and outputs from DPDECODE to control additional MUXes in the datapath. These MUXes can (only for the MOVCn instructions you decide to implement select the output of your hardware block(s) instead of the ALU. You can put your hardware on a separate sheet which you add as a component to the datapath sheet.

Use hierarchy to simplify your hardware design (this will also speed up design and testing!). As an example, look at theshift block you are given as part of EEP1. Be creative: you do not have to follow the examples in the notes exactly. Reuse hardware whenever possible when two different and related operations need to be implemented.

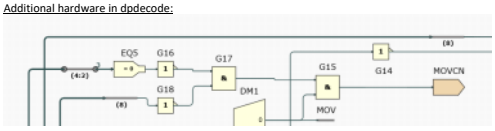
Additional hardware in the datapath:



Comments:

When c field = 1 and we have MOVCN = 1, G2.output = 1 and the multiplier starts running. During the time when the multiplierstarts, whether MOVCN is high does not matter as it is then controlled by the counts of counter in multiplier.

Additional hardware in dpdecode:



Comments:

EQ5 detects whether the c field is 0 and whether bit 8 is 0 in the instruction:

=> because if bit 8 is 1, it means the c field is not 0 because of the immediate value in instruction

=> MOVCN cannot be 1 then

If we have a MOV instruction, bit 8 is not 1, and the c field is not empty => MOVCN = 1

5.2.3 Using your new hardware

Try to work out optimal sequences of instructions. Consider whether slightly different hardware could speed up the software.

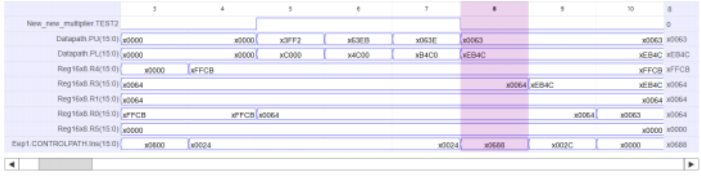
lab2_challenge.ram

```
lab2 > ≡ lab2_challenge_code1.ram
1   0x00 0x01cb
2   0x01 0x0364
3   0x02 0x0620
4   0x03 0x0880
5   0x04 0x0024
6   0x05 0x0024
7   0x06 0x0024
8   0x07 0x0024
9   0x08 0x0688
10  0x09 0x002c
11
```

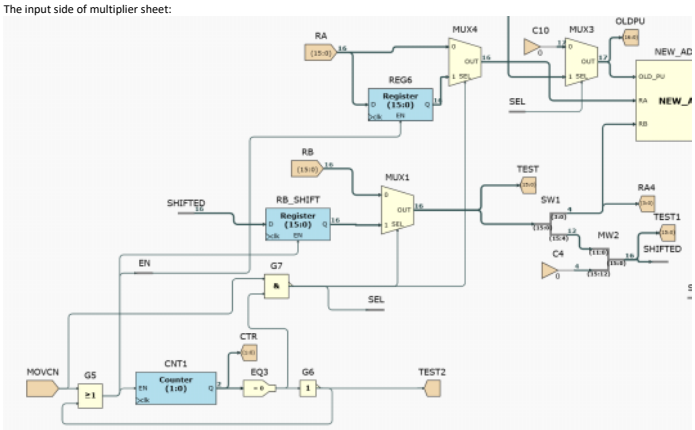
lab2_challenge_code1.txt

```
lab2 > ≡ lab2_challenge_code1.txt
1   MOV R0,#203
2   MOV R1,#100
3   MOV R3, R1
4   MOV R4, R0
5
6   MOVC1 R0, R1
7   MOVC1 R0, R1
8   MOVC1 R0, R1
9   MOVC1 R0, R1
10  MOVC2 R3, R4
11  MOVC3 R0, R1
```

Waveform simulation:



The input side of multiplier sheet:



When the CPU first implements the MOVCN instruction, MOVCN = 1 at clock cycle 4. This enables the counter, making it start counting from 0. But when the instruction is implemented, at the first clock cycle, counter value is 0 so EQ3 = 1. Hence, MOVCN "AND"ed with EQ3 is 1, making G7 output 0. Hence, this selects port 0 of MUX1. Hence, at clock cycle 4, immediately when MOVC1 R0, R1 is executed, the least significant 4 bits are shifted out at the same clock cycle as shown by viewer RA4.

At the same time MOVCN is 1 and counter value is not 0, the register RB_SHIFTED that stores the 16-bit right-shifted number of Rb is enabled at clock cycle 4. Hence at clock cycle 5, the register value is updated to be x0006. Since EQ3 is 0 with counter value no longerat 0, the output of G7 is 1 and port 1 - RB_SHIFTED.Q is selected by MUX1.

The multiplication runs for 4 clock cycles (from clock cycle 5 to clock cycle 8). With my .ram file keeping the MOVCN signal high with the instructions, at clock cycle 8, the counter value becomes 0 after counting from 0(cycle 4) to 1(cycle 5) to 2(cycle 6) to 3(clock cycle 7). This makes G7 output 0 at clock cycle 8

At clock cycle 8 is when instruction MOVC2 R3, R4 is executed, so from the datapath sheet additional hardware design, the PLvalue xEB4C is written to R3 which as you can see in waveform simulation is updated to xEB4C at clock cycle 9.

In the code, I used 4 MOVC1 instructions as placeholder to ensure that the CPU only writes the finished and correct value ofPL and PU to registers when MOVC2 and MOVC3 instructions are executed.

Lab2_challenge2.txt

```
lab2_challenge2.txt
MOV R0,#203
MOV R1,#100
MOV R3, R1
MOV R4, R0

MOVC1 R0, R1
MOVC1 R0, R1
MOVC1 R0, R1
MOVC1 R0, R1
MOVC2 R3, R4
MOVC3 R0, R1

MOV R5, #5

EXT 0xA1
MOV R0, #23
MOV R1, #60
MOV R3, R1
MOV R4, R0

MOVC1 R0, R1
MOVC1 R0, R1
MOVC1 R0, R1
MOVC1 R0, R1
MOVC2 R3, R4
MOVC3 R0, R1
```

I also tested the code above and got the following waveform simulation:



Conclusion:
Overall, I managed to implement unsigned multiplication in fewer clock cycles than in Task 3 and 4 in lab2. However, from mysimulations, my implementation can only handle unsigned number multiplication. Hence, there is still much room for improvement.

5.3 Collaboration: IEEE 754 half-precision (binary16) Floating Point

This option has never yet been attempted and is challenging but achievable by two students working as a pair: one on softwareand the other on hardware.

For additional credit, optionally combine hardware and software to make a fast IEEE-754 binary16 multiply routine. This is quite a challenge. To simplify the task, you may assume that *subnormal* and *special* (infinite or NaN) numbers do not occur as input or output. The standard normalised sign-magnitude-exponent coding described in the reference is thus always used together with, as a single special case, zero. IEEE-754 specifies detailed rounding requirements.

For this challenge, you need not implement correct rounding as long as the result is one of the two closest possible floating-point numbers to the exact value.

For your interest, a complete floating-point library would use this multiply routine and similar routines for addition and subtraction. Division (or more accurately, reciprocal from which division can be calculated) can be implemented using multiplication and a Newton-Raphson algorithm. Such a complete library can therefore be implemented quite efficiently in software given only a fast multiply algorithm.